Sistemas multiagente para Parachute S.A.

Comparación de orquestación centralizada, jerárquica y descentralizada

HOJA 5

Clima, seguridad de salto, agenda y FAQs

Objetivo: resolver el mismo flujo funcional con tres formas de colaboración entre agentes, conservando una única implementación de las integraciones.

1. Solución común

Los tres programas comparten seis especialistas: triaje, extracción de fecha, clima, seguridad, agenda y FAQs. WeatherService consulta Open-Meteo con las coordenadas 14.013722, -90.771611 y solicita temperatura máxima, precipitación acumulada, cobertura media de nubes, viento máximo y ráfagas máximas. La fecha debe estar entre hoy y el día 16 de la ventana, ambos inclusive.

Reglas de aceptación

• Ideal: viento menor a 20 km/h, sin lluvia y nubes menores a 30%.

• Marginal: viento entre 20 y 28 km/h o nubes entre 30% y 75%.

• El viento marginal exige que el usuario declare un tándem experimentado.

• Prohibido: viento mayor a 28 km/h, ráfagas mayores a 35 km/h, cualquier precipitación o nubes mayores a 75%.

Abstracciones para crecimiento

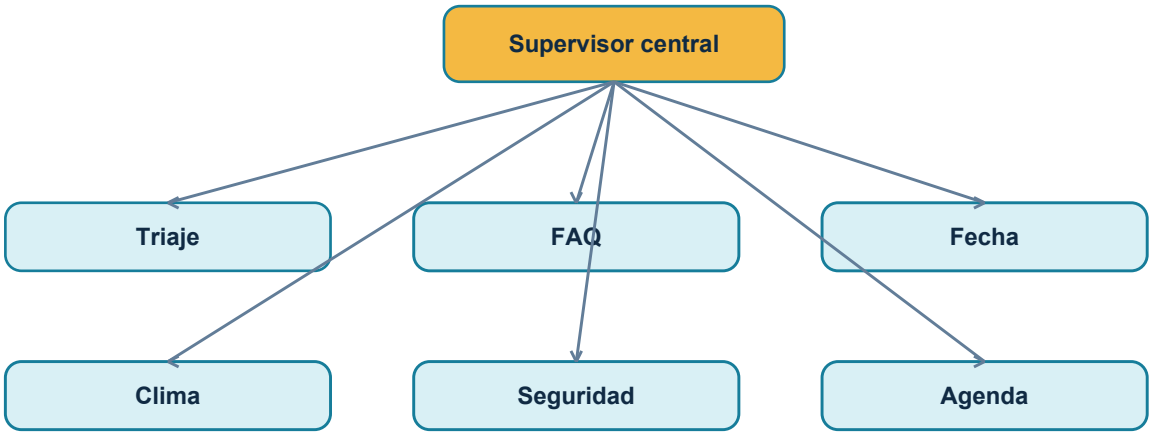
WeatherService, assess_weather y AppointmentRepository no conocen la arquitectura de orquestación. factory.build_specialists compone una vez las integraciones y entrega los mismos objetos a cada programa. Por eso un nuevo canal, regla o proveedor puede incorporarse sin reescribir las tres variantes.

Decisión técnica

Las reglas que autorizan un salto son deterministas. Un LLM no decide si una condición es segura ni puede omitir una validación. El RAG anterior se conserva para FAQs; los handoffs de la variante descentralizada son explícitos y auditables. Esto mantiene compatibilidad con el cliente NVIDIA/OpenAI existente.

2. Arquitectura centralizada

CentralSupervisor clasifica la solicitud y controla directamente toda la secuencia. Cada resultado vuelve al supervisor antes de invocar al siguiente especialista.



Fortaleza principal	Flujo simple, trazabilidad directa y un único lugar para políticas globales.
Limitación principal	El supervisor concentra conocimiento y se convierte en cuello de botella a medida que crecen los dominios.

3. Arquitectura jerárquica

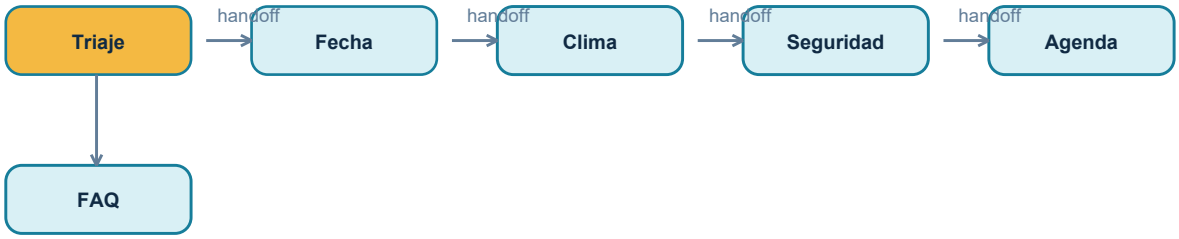
ExecutiveManager delega por dominio. InformationManager atiende FAQs y OperationsManager coordina fecha, clima, seguridad y agenda.



Fortaleza principal	Escala mejor: cada manager encapsula un dominio y reduce la complejidad del nivel raíz.
Limitación principal	Agrega capas, más objetos y una ruta de coordinación un poco más larga.

4. Arquitectura descentralizada

PeerNetwork no tiene supervisor. Triaje entrega la solicitud al primer par y cada agente hace handoff al siguiente usando el contexto acumulado.



Fortaleza principal	Bajo acoplamiento con un coordinador central y fácil sustitución de pares especializados.
Limitación principal	Es más difícil observar el flujo completo, evitar ciclos y aplicar políticas transversales.

5. Respuestas

1. ¿Qué arquitectura resuelve mejor este problema? ¿Por qué?

La arquitectura jerárquica ofrece el mejor equilibrio para este caso. Hoy el flujo tiene dos dominios claros: información (FAQs) y operaciones (clima, seguridad y agenda). Un manager ejecutivo mantiene una interfaz única, mientras managers de dominio absorben el crecimiento anunciado por Parachute S.A. sin convertir al supervisor raíz en una clase monolítica. También facilita aplicar una política de seguridad común y observar quién autorizó cada reserva.

La centralizada también es una buena opción para el alcance actual: es la más simple de operar y depurar. Sin embargo, pierde atractivo cuando aumentan las funciones. La descentralizada demuestra handoffs correctamente, pero no es la primera elección en un proceso corto, secuencial y regulado, porque la gobernanza, la trazabilidad y el control de errores requieren trabajo adicional.

2. ¿Es necesario utilizar un sistema multiagente? ¿Por qué?

No es estrictamente necesario. Un servicio convencional con módulos de RAG, clima, reglas y persistencia resolvería el requisito con menor costo operativo, menos latencia y pruebas más sencillas. Además, las reglas meteorológicas son deterministas y no se benefician de razonamiento autónomo.

Sí puede ser justificable como diseño evolutivo y como ejercicio académico. La separación por agentes crea límites claros de responsabilidad y permite que, en el futuro, reservas, soporte, pagos, notificaciones y logística evolucionen o escalen de manera independiente. La recomendación práctica es conservar las integraciones y políticas críticas como servicios deterministas, usando agentes para coordinación, lenguaje natural y delegación, no para reemplazar controles de seguridad.

Conclusión

Recomendación: arquitectura jerárquica para producción; centralizada para un MVP; descentralizada cuando existan dominios realmente autónomos.

Referencias

Open-Meteo Forecast API: https://open-meteo.com/en/docs
OpenAI Agents - manager agents as tools y handoffs: https://openai.github.io/openai-agents-python/agents/